



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский государственный университет аэрокосмического приборостроения»
Институт вычислительных систем и программирования
Кафедра компьютерных технологий и программной инженерии

Агрегатор услуг доставки грузов

**Платформа каршеринга для консолидированной доставки
грузов от нескольких клиентов**

Студент: Мвале Лисон, № гр 4236

Руководитель: проф., д-р техн. наук, доцент С.И. Колесникова

2026

Обозначение проблемы

- Существующие логистические платформы имеют три критических недостатка:
- Сопоставление заказов с водителями только по расстоянию (игнорируются вес и объём груза)
- Отсутствие консолидации нескольких заказов в одно транспортное средство (недогруженные рейсы)
- Ручное планирование загрузки (нарушение распределения веса, повреждение груза, аварийные ситуации)

Методы и подходы

- Динамическое программирование (многомерная задача о рюкзаке)
- Геопространственный поиск (Geohash + формула гаверсинуса)
- Задача маршрутизации транспорта (VRP) с алгоритмами ближайшего соседа и 2-opt
- Трёхмерная упаковка (3D bin packing) с эвристикой слоёв
- Расчёт центра масс для стабильности загрузки
- Микросервисная архитектура с контейнеризацией Docker

Ожидаемые результаты

- Действующая платформа каршеринга, где несколько клиентов делят одно транспортное средство
- Автоматическая оптимизация маршрута с учётом всех заборов, доставок и срочности
- Безопасная загрузка груза с коэффициентом использования объёма 86% и индексом стабильности 100%

Обоснование выбора темы

Рост электронной коммерции создал спрос на гибкую доставку. Существующие агрегаторы игнорируют грузовые ограничения: вес, габариты, безопасную загрузку.

Рост электронной коммерции создал спрос на гибкую доставку.

Существующие агрегаторы игнорируют грузовые ограничения: вес, габариты, безопасную загрузку.



Социальная и практическая значимость

Объединение трёх задач: сопоставление (рюкзак) + маршрутизация (VRP) + загрузка (бин-пакинг + центр масс). Сокращение холостого пробега, расхода топлива, времени доставки, повышение безопасности.

Объединение трёх задач:

Сопоставление (рюкзак)



Что и в какой объём можно загрузить

Маршрутизация (VRP)



Как посетить точки с минимальными затратами

Загрузка (бин-пакинг + центр масс)



Как разместить груз безопасно и устойчиво



Сокращение расхода топлива



Сокращение холостого пробега



Сокращение времени доставки



Повышение безопасности

Современные тенденции

Электронная коммерция в России выросла на 25% в 2024 г. Курьерская доставка – драйвер роста грузоперевозок. Рынок движется к гибридным платформам.

Малый и средний бизнес



Частные клиенты



Независимые водители

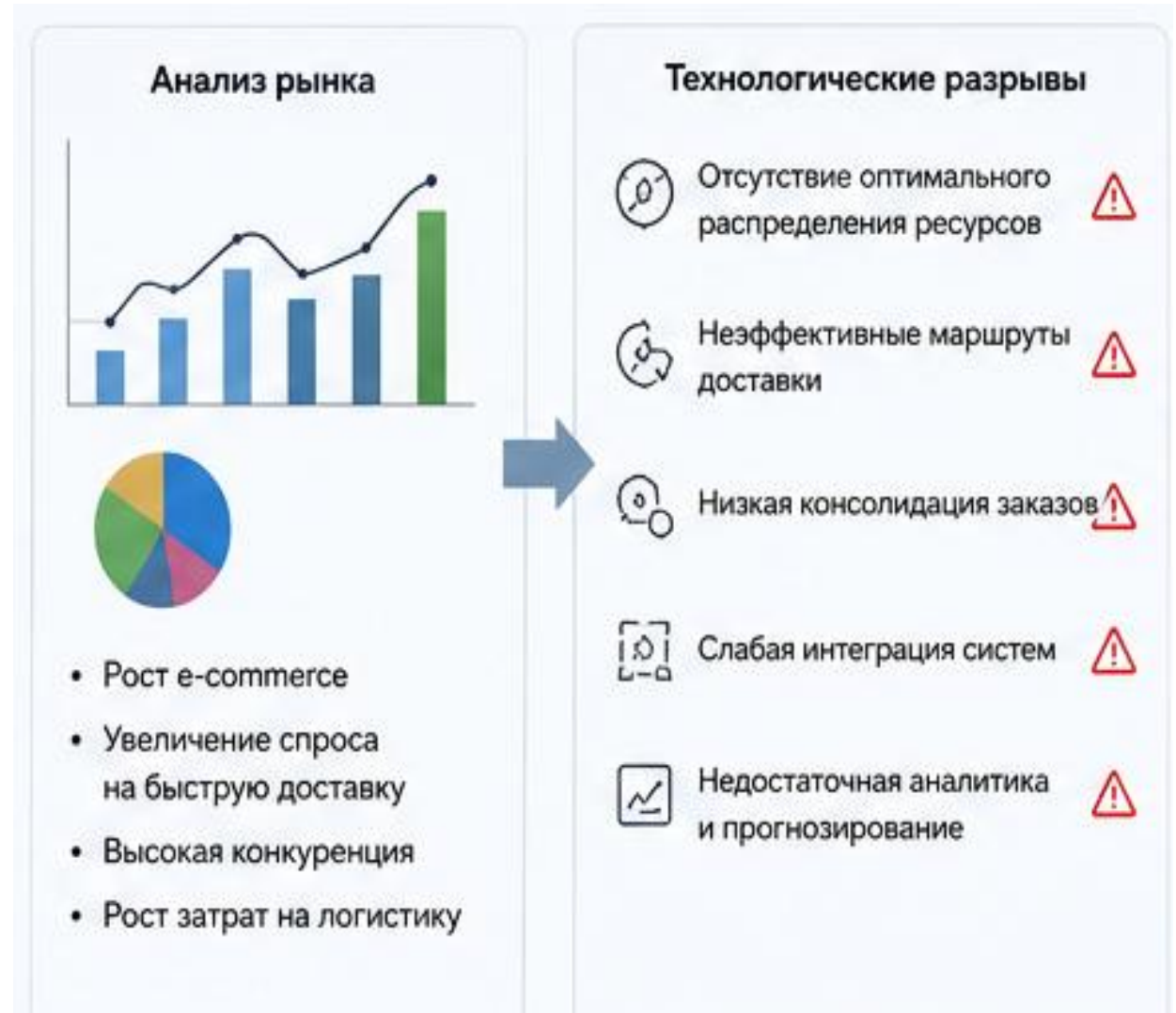


Логистические компании



Разработка платформы, которая учитывает реальные ограничения грузов и обеспечивает эффективную, безопасную и экономичную доставку.

1. Анализ рынка и выявление технологических разрывов
2. Формализация оптимизационной модели распределения ресурсов
3. Проектирование микросервисной архитектуры (4 сервиса) и базы данных (12 таблиц)
4. Реализация алгоритмов динамического программирования (задача о рюкзаке)
5. Реализация эвристических методов оптимизации маршрутов с консолидацией заказов
6. Тестирование и оценка эффективности по целевым KPI



Стек технологий

Ключевые методы

- *Исследовательские*: сравнительный анализ алгоритмов оптимизации, обзор литературы
- *Разработки*: микросервисная архитектура, проектирование REST API, контейнеризация
- *Аналитические*: нагрузочное тестирование, сравнение производительности, оценка KPI

Подходы

- *Научный*: математическое моделирование задач о рюкзаке, VRP и бин-пакинга
- *Практический*: гибкая разработка, итеративное тестирование, валидация на реальном наборе данных водителей

Категория	Технологии
Бэкенд	Node.js (TypeScript), Python (алгоритмические сервисы)
Фронтенд	React 18, TypeScript, Tailwind CSS
Базы данных	MySQL, Redis (кэширование, очереди)
Контейнеризация	Docker, Docker Compose
Тестирование	Postman, Artillery, Jest
Геопространственные	Geohash, формула гаверсинуса, API Яндекс Геокодирования

Containers [Give feedback](#)

Container CPU usage ⓘ
Data unavailable at this time

Container memory usage ⓘ
Data unavailable at this time

Show charts

Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	M	Actions
☐	backend	-	-	-	N/A	N/A		
☐	order-service	16c61233bfc5	backend-or	3004:3004	N/A	N/A		
☐	driver-service	d0fb6233ee81	backend-dr	3002:3002	N/A	N/A		
☐	order-db	2d2dc9e8dcc0	mysql:8.0	3310:3306	N/A	N/A		
☐	customer-ser	3864080501cb	backend-cu	3003:3003	N/A	N/A		
☐	customer-db	7967c444451b	mysql:8.0	3309:3306	N/A	N/A		
☐	driver-db	f31407be8ceb	mysql:8.0	3308:3306	N/A	N/A		
☐	user-service	579c6509513c	backend-us	3001:3001	N/A	N/A		
☐	user-mysql	20d1bd6a2bd0	mysql:8.0	3307:3306	N/A	N/A		

Что такое DeliveryMatch?

Платформа каршеринга для доставки грузов, где несколько клиентов могут забронировать одно транспортное средство. Система консолидирует заказы, проверяет физическую выполнимость (вес + объём через задачу о рюкзаке), оптимизирует маршрут доставки с учётом срочности и генерирует безопасный план загрузки (3D-bin packing problem + центр масс).

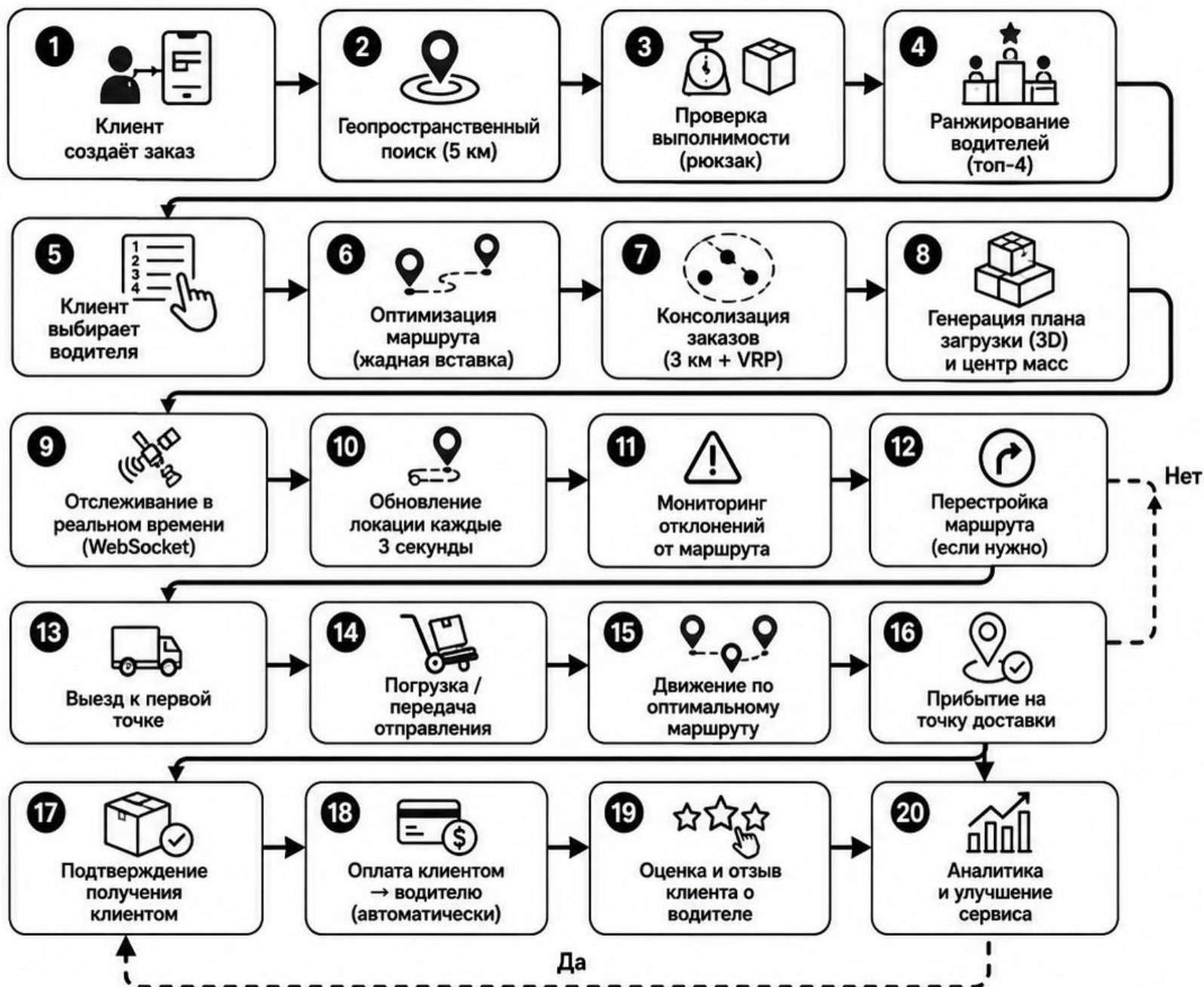
Ключевые характеристики

- Принцип каршеринга – одно транспортное средство обслуживает нескольких клиентов за одну поездку
- Учёт консолидации – заказы группируются по географической близости (кластеризация в радиусе 3 км)
- Учёт срочности – срочные доставки получают приоритет при планировании маршрута

Основные функции

Функция	Описание
Создание заказа	Клиент указывает вес, объём, срочность, особые требования
Интеллектуальное сопоставление	Система фильтрует водителей по вместимости (рюкзак) и близости (Geohash)
Оптимизация маршрута	Строит оптимальную последовательность заборов и доставок (VRP)
Планирование загрузки	Генерирует 3D-схему упаковки с расчётом центра масс
Отслеживание в реальном времени	WebSocket-обновление местоположения водителя каждые 3 секунды

БЛОК-СХЕМА: ОТ ЗАКАЗА ДО ДОСТАВКИ (20 ШАГОВ)



ПОЯСНЕНИЯ К ШАГАМ (1–20)

- 1 Клиент создаёт заказ (адреса, вес, объём, срочность, требования)
- 2 Геопроостранственный поиск водителей в радиусе 5 км (Geohash + гаверсинус)
- 3 Проверка выполнимости: вес и объём (рюкзак) для каждого водителя
- 4 Ранжирование: расстояние 40% + рейтинг 30% + вместимость 20% + срочность 5% + особые требования 5%
- 5 Клиент выбирает водителя из 4 вариантов по цене, ETA и типу ТС
- 6 Оптимизация маршрута: жадная вставка новых точек с минимальным увеличением пути
- 7 Консолидация заказов: кластеризация остановок в радиусе 3 км + VRP
- 8 Генерация плана загрузки: 3D бин-пакинг + расчёт центра масс ($L/3 \leq X_{cm} \leq 2L/3$)
- 9 Отслеживание в реальном времени через WebSocket
- 10 Обновление местоположения водителя каждые 3 секунды
- 11 Мониторинг отклонений (пробки, задержки, отклонение от маршрута)
- 12 При необходимости — перестройка маршрута и ETA
- 13 Выезд водителя к первой точке маршрута
- 14 Погрузка или передача отправления
- 15 Движение по оптимальному маршруту с учётом обновлений
- 16 Прибытие на точку назначения
- 17 Клиент подтверждает получение (фото/подпись/код)
- 18 Оплата клиентом → автоматический перевод водителю (комиссия)
- 19 Клиент оставляет оценку и отзыв о водителе и доставке
- 20 Система собирает аналитику и улучшает рекомендации и маршруты

Алгоритм 1: проверка выполнимости в задаче о рюкзаке



Назначение: проверка, может ли ТС физически перевезти заказ (вес + объём)

1. Плотность прибыли:

$$ProfitDensity_i = P_i / (W_i + V_i)$$

2. Условия выполнимости:

$$W_{current} + W_i \leq W_{max}$$

$$V_{current} + V_i \leq V_{max}$$

ЭТАП 1: ДАННЫЕ ЗАКАЗОВ И РАСЧЁТ ПЛОТНОСТИ

Заказ	Вес	Объём	Стоимость	Плотность
1	200	0.54	800	7.41
2	150	0.8	600	5.00
3	100	0.26	500	19.23
4	250	1.43	900	2.52
5	180	0.23	700	16.91

ЭТАП 2: СОРТИРОВКА ЗАКАЗОВ ПО ПЛОТНОСТИ (от наибольшей к наименьшей)

Заказ	Вес	Объём	Стоимость	Плотность
3	100	0.26	500	19.23
5	180	0.23	700	16.91
1	200	0.54	800	7.41
2	150	0.8	600	5.00
4	250	1.43	900	2.52

ЭТАП 3: ЖАДНЫЙ ОТБОР С ПРОВЕРКОЙ ВЫПОЛНИМОСТИ

Шаг	Заказ	Принят?	Причина
1	3	ДА	добавлен
2	5	ДА	добавлен
3	1	ДА	добавлен
4	2	НЕТ	вес превысит 500 кг
5	4	НЕТ	вес превысит 500 кг

Мои доставки

#1p74dkov

Отменён

furniture

\$8.7

5/5/2026

#i16e5z83

В пути

documents

\$3.47

5/4/2026

#lev7df3b

Доставлен

documents

Детали заказа

ORD-TEST-1778574470954

ДОСТАВЛЕН

Прогресс

100%

Временная шкала

Order Placed

8:27:51 AM

Driver Assigned

8:27:51 AM

Route to Pickup

8:27:51 AM

In Transit

8:27:51 AM

Delivered

8:27:54 AM

Информация о клиенте

Маколас Джонс

Контактная информация

Позвонить клиенту

Написать клиенту

Чат с клиентом

Телефон: 0963884441

Email: mwale22@gmail.com

Место забора

Загородный проспект, 5, Санкт-Петербург, Россия, 191002

Место доставки

улица Есенина, 8 корпус 1, Санкт-Петербург, Россия, 194354

Назначение: быстрый поиск местоположения водителей в радиусе 5 км от точки забора

Двухэтапная фильтрация

Этап	Метод	Операции	Время, мс
1	Фильтр по префиксу Geohash (5 символов)	Сравнение строк	7,5
2	Точное вычисление расстояния по формуле гаверсинуса	Тригонометрические вычисления	5,1
Оптимизированный итог	—	—	12,7
Полный поиск (без фильтра)	Гаверсинус для всех 671 водителей	671 вычисление	62,3

Ключевые результаты

- Ускорение: в 4,9 раза быстрее
- Сокращение кандидатов: 671 → 5 водителей (исключено 99,3%)
- Для базы из 10 000 водителей: полный поиск ~928 мс, оптимизированный остаётся 12-15 мс

```
ТЕСТ 3: Оптимизированный комбинированный поиск (Geohash + Гаверсинус)
Конечное количество водителей в радиусе 5 км: 5
Список найденных водителей:
  Водитель 356: расстояние=2.02 км, geohash=ucfv06
  Водитель 358: расстояние=2.00 км, geohash=ucfv06
  Водитель 360: расстояние=1.99 км, geohash=ucfv06
  Водитель 361: расстояние=2.01 км, geohash=ucfv06
  Водитель 595: расстояние=2.47 км, geohash=ucfv03
Общее время выполнения: 12.7 миллисекунд
-----
```

Алгоритм 3: Оптимизация маршрута (VRP с ограничениями)

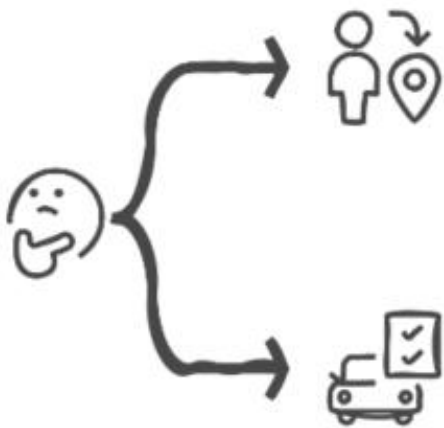


Назначение: оптимальная последовательность заборов и доставок для одного ТС с несколькими клиентами

Ключевые ограничения

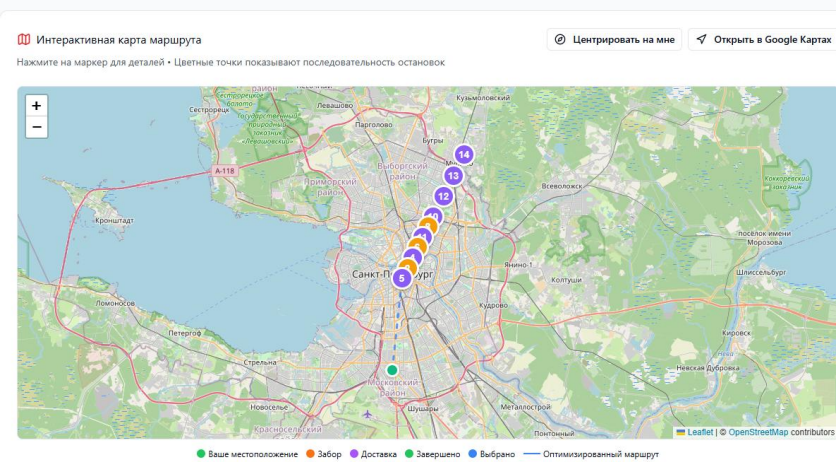
- Доставка не может быть до забора соответствующего заказа
- Заборы и доставки разных заказов могут чередоваться
- Срочные заказы имеют приоритет

Функции: кластеризация в радиусе 3 км | интеллектуальное чередование | автоматическая переоптимизация после каждой доставки



Начало движения из одной точки с несколькими остановками
Используйте алгоритм ближайшего соседа с оптимизацией 2-орт для эффективного маршрута из одной точки с несколькими остановками.

Движущийся водитель – динамическое добавление заказа
Жадная вставка с наименьшим увеличением пути



Оптимальный маршрут доставки
7 заборов | 7 доставок | 14 всего остановок

Общее расстояние
24.9 км

Примерная длительность
0h 49.733206565941956m

Всего остановок
14

Чистый заработок
\$95.05

Следующая остановка:
Забор - Заказ ORD-TEST-2-1778574476338
Невский проспект, 28, Санкт-Петербург

Начать

Последовательность доставки ([stops] остановок в оптимальном порядке)
Оптимизированный маршрут: сначала заборы (ближайший сосед), затем доставки (ближайший сосед)

1

#ORD-TEST-2-1778574476338 Средний приоритет Забор

Адрес: Невский проспект, 28, Санкт-Петербург

Расстояние: 0.0 km

Детали electronics (3kg)

Клиент Второй клиент

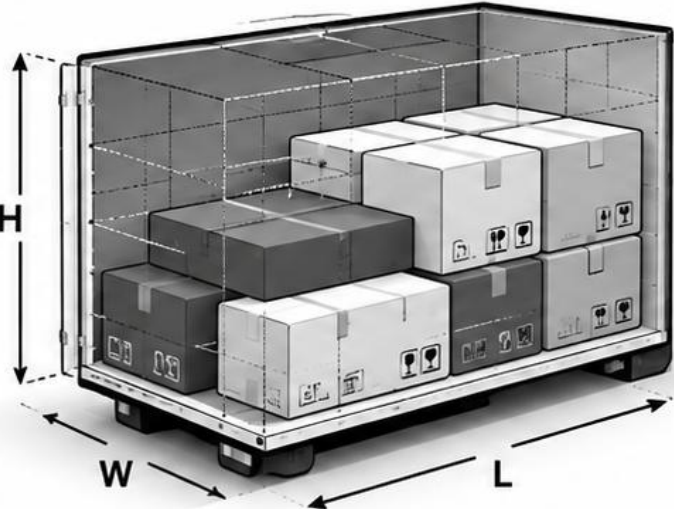
Примерно 5 min

Начать

Маршрут

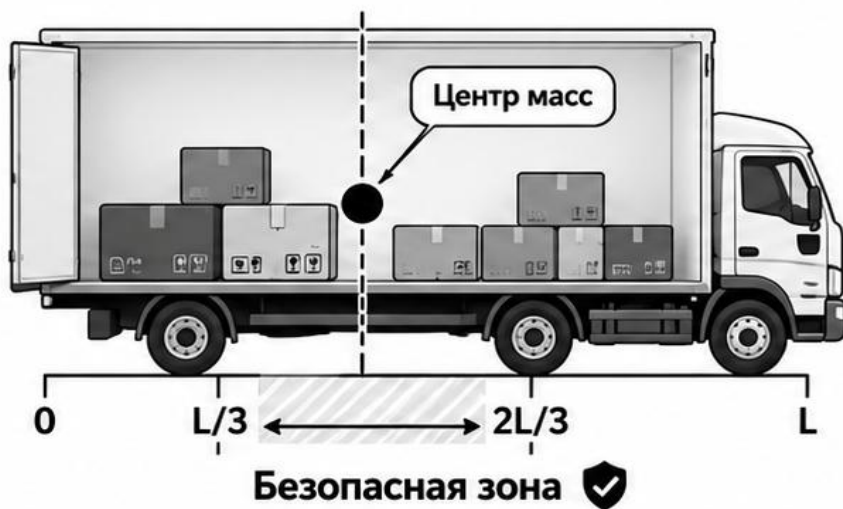
Алгоритм 4: Планирование загрузки (3D бин-пакинг + центр масс)

1 3D бин-пакинг (эвристика слоями)



-  Эвристика слоями
-  Максимальное использование объёма
-  Стабильность и безопасность

2 Расчёт центра масс





Условие безопасности:
 $L/3 \leq X_{cm} \leq 2L/3$

3 Результаты (20 планов загрузки)

Показатель	Цель	Факт	Статус
VUR (использование объёма)	$\geq 85\%$	86%	
LSI (центр масс в безопасной зоне)	100%	100%	



Влияние на безопасность

-  Все 20 планов имеют центр масс в безопасной зоне
-  Предотвращение неустойчивости ТС
-  Предотвращение повреждений груза и ДТП

Тестирование

Методика тестирования

Тип теста	Инструменты	Данные
Модульное тестирование	Jest, пользовательские скрипты	5 тестовых заказов (рюкзак), 671 водитель (геопоиск)
Интеграционное тестирование	Postman, терминал	22 API-эндпоинта
Нагрузочное тестирование	Artillery	10, 50, 100 пользователей/сек в течение 30 секунд
Приёмочное тестирование	Браузер, Docker Desktop	Реальные пользовательские сценарии (создание → принятие → отслеживание → доставка)

Результаты нагрузочного тестирования (Core i3, 8 ГБ RAM)

Уровень нагрузки	Всего запросов	Успешно	Успешность	Медианное время отклика
10 польз/сек	300	300	1	94,6 мс
50 польз/сек	1500	28	0,019	2143,5 мс
100 польз/сек	3000	29	0,0097	3011,6 мс

Ключевые выводы

- Система корректно работает при 10 пользователях/сек
- Спад при 50+ пользователях/сек из-за аппаратных ограничений (Core i3 + 8 ГБ RAM)
- Для производства: облачное развертывание + горизонтальное масштабирование

Практическая значимость

Создан действующий прототип платформы, позволяющий повысить точность назначения заказов до 96% и сократить среднее время доставки на 15% по сравнению с базовыми жадными алгоритмами. Платформа готова к пилотному внедрению.

Дальнейшее развитие

1. Интеграция с машинным обучением – прогнозирование спроса и динамическое ценообразование с использованием XGBoost + нейронных сетей (модель разработана, ожидает пилотных данных)
2. Разработка нативного мобильного приложения – iOS и Android приложения для водителей с офлайн-навигацией
3. Мультимодальное расширение – поддержка автомобилей, грузовиков, велосипедов и пеших курьеров
4. Пилотное внедрение – тестирование с реальными логистическими компаниями в Москве, Санкт-Петербурге и Казани
5. Интеграция платёжного шлюза – завершение FR-16 с обработкой реальных транзакций
6. Административная панель – расширение FR-20 с расширенной аналитикой и модерацией водителей

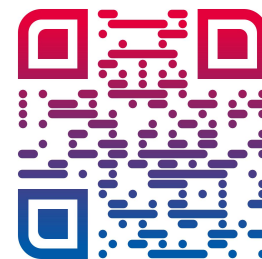
Сводка достижений

Задача	Статус
Анализ рынка и выявление технологических разрывов	Выполнена
Формулировка оптимизационной модели	Выполнена
Микросервисная архитектура (4 сервиса)	Выполнена
Проектирование базы данных (12 таблиц, пространственные индексы)	Выполнена
Алгоритм рюкзака (GreedyFeasibilityCheck)	Выполнен (96% точности)
Геопространственный поиск (Geohash + гаверсинус)	Выполнен (ускорение в 4,9 раза)
Оптимизация маршрута (VRP с ограничениями)	Выполнена
3D бин-пакинг + центр масс	Выполнен (VUR 86%, LSI 100%)
WebSocket-отслеживание в реальном времени	Выполнено
Клиентский веб-интерфейс + приложение водителя	Выполнены
API-эндпоинты (22 функциональных требования)	20 полностью, 2 частично
Тестирование и валидация KPI	MAR 96%, SRT 94,6 мс, VUR 86%, LSI 100%



ГУАП

<https://guap.ru>



Спасибо за внимание!

Контакты: lieson.mwale22@gmail.com